EEPROM Redundant Read with Fallback

PROJECT DOCUMENTATION

Tool:

- S32 Design Studio
- Putty(64bit)

Interfaces:

- I2C EEPROM
- UART (PuTTY)

1. Overview

This project shows how a microcontroller can protect a stored value against corruption. A value is saved in EEPROM twice: a **main copy (Copy 1)** and a **backup copy (Copy 2)**. Each copy has a **CRC** (a check code calculated from the data). When the program starts, it checks Copy 1. If Copy 1 is damaged, it uses the backup Copy 2 to repair it. Everything it does is printed over UART so it can be watched in PuTTY.

Final result of every run is one of three status codes:

Status	Value	Meaning in plain words
NVM_REQ_OK	0	Copy 1 was fine. Nothing to fix.
NVM_REQ_RESTORED_FROM_ROM	1	Copy 1 was damaged, Copy 2 was fine, so Copy 1 was repaired from Copy 2.
NVM_REQ_INTEGRITY_FAILED	2	The data cannot be trusted (flag off or invalid, read/write error, or both copies bad).

2. Requirements and scope

R_n	Requirement
R1	When ROM_BLOCK = 1, check the CRC of Copy 1.
R2	If the Copy 1 CRC fails, call NvM_ReadMirrorCopy() to fetch Copy 2 and report NVM_REQ_RESTORED_FROM_ROM.
R3	If ROM_BLOCK = 0, report NVM_REQ_INTEGRITY_FAILED.

R_n	Requirement
R4	Initial values of ROM_BLOCK, Copy 1 and Copy 2 are written by the provisioning function, then verified over UART.
R5	Keep the scope small: only **2 fixed blocks** (Copy 1 and Copy 2), not a full configuration-driven table.

3. How it works (simple view)

1. Power on and initialize clock, pins, UART, I2C and CRC.
2. Read the safety flag `ROM_BLOCK`. It must be exactly 1, otherwise stop with an error.
3. Read Copy 1 data and its stored CRC.
4. Recalculate the CRC from the Copy 1 data and compare it with the stored CRC.
5. If they match, report `NVM_REQ_OK`.
6. If they do not match, read Copy 2 and run the same check on it.
7. If Copy 2 is also bad, report `NVM_REQ_INTEGRITY_FAILED`.
8. If Copy 2 is good, write its data and CRC over Copy 1 and report `NVM_REQ_RESTORED_FROM_ROM`.

4. EEPROM memory map

All addresses are byte addresses inside the EEPROM. Each data item is written in a single write that stays inside one 16-byte page, as the driver requires.

Item	Address	Size	Description
Copy 1 data	0x0000	4 bytes (uint32)	Main stored value
Copy 1 CRC	0x0004	4 bytes (uint32)	Check code stored with Copy 1
Copy 2 data	0x0010	4 bytes (uint32)	Backup (mirror) of the value
Copy 2 CRC	0x0014	4 bytes (uint32)	Check code stored with Copy 2
ROM_BLOCK	0x0020	1 byte (uint8)	Safety flag: 1 = fallback allowed

5. Test data written by provisioning

Provisioning is controlled by ENABLE_EEPROM_PROVISIONING. The values below are the defaults in the code.

Item	Macro	Value written	Notes
ROM_BLOCK	INITIAL_ROM_BLOCK	1	Fallback enabled
Copy 1 data	INITIAL_COPY1_DATA	67892	
Copy 1 CRC	(calculated)	CRC of **12345**	Deliberately wrong for 67892, so Copy 1 fails its check
Copy 2 data	INITIAL_COPY2_DATA	12345	
Copy 2 CRC	(calculated)	CRC of 12345	Correct

The fault is created on purpose: Copy 1's CRC is calculated from INITIAL_COPY2_DATA instead of its own data. This simulates a corrupted main copy.

6. Software structure

File	Function	Purpose
main.c	main()	Initializes peripherals, optionally provisions EEPROM, runs the test, prints the final status, then idles.
main.c	UART_Print()	Sends a string over UART and waits until transmission finishes.
main.c	CalculateCRC()	Re-initializes the CRC module and returns the CRC of a 32-bit value.
main.c	EEPROM_Read32()	Reads a 32-bit value from EEPROM (null-pointer safe).
main.c	ProvisionEEPROMData()	Writes ROM_BLOCK, Copy 1 (with wrong CRC) and Copy 2 (with correct CRC).
main.c	NvM_ReadMirrorCopy()	Reads Copy 2 data and CRC and validates them.
main.c	RedundantReadTest()	Main algorithm: flag check, Copy 1 check, fallback and restore.

File	Function	Purpose
eeprom.c	EEPROM_ReadBytes()	I2C read: send word address, then receive bytes.
eeprom.c	EEPROM_WriteBytes()	I2C page-safe write, then a 5 ms wait for the write cycle.
eeprom.h	declarations	Public prototypes of the two EEPROM functions.

EEPROM driver rules (eeprom.c)

Item	Value / rule
I2C slave address	0x50
I2C timeout	100 ms (EEPROM_TIMEOUT)
Address space	256 bytes (address + length must not exceed 256)
Max write length	16 bytes per call
Page rule	A write must not cross a 16-byte page boundary
Write cycle delay	5 ms after each write (OSIF_TimeDelay(5U))
Errors	Returns STATUS_ERROR for a null pointer, zero length or an invalid range

7. Algorithm decision table

Condition checked	Result	UART message	Returned status
ROM_BLOCK read fails	Stop	ERROR: ROM_BLOCK read failed	NVM_REQ_INTEGRITY_FAILED
ROM_BLOCK = 0	Stop	ROM_BLOCK is 0	NVM_REQ_INTEGRITY_FAILED
ROM_BLOCK is not 0 and not 1	Stop	ROM_BLOCK is invalid	NVM_REQ_INTEGRITY_FAILED
Copy 1 data or CRC read fails	Stop	ERROR: Copy 1 data read failed / Copy 1 CRC read failed	NVM_REQ_INTEGRITY_FAILED
Copy 1 calculated CRC = stored CRC	Done	Copy 1 CRC OK	NVM_REQ_OK

Condition checked	Result	UART message	Returned status
Copy 1 CRC mismatch	Go to Copy 2	Copy 1 CRC FAILED, Reading mirror copy...	(continues)
Copy 2 read fails or its CRC mismatches	Stop	ERROR: Mirror ... read failed or Copy 2 CRC FAILED	NVM_REQ_INTEGRITY_FAILED
Copy 2 is good, restore of Copy 1 data or CRC fails	Stop	ERROR: Copy 1 restoration failed / Copy 1 CRC restoration failed	NVM_REQ_INTEGRITY_FAILED
Copy 2 is good, restore succeeds	Done	Copy 1 restored from Copy 2	NVM_REQ_RESTORED_FROM_ROM

8. Configuration

8.1 Compile-time switches in main.c

Macro	Default	Purpose
ENABLE_EEPROM_PROVISIONING	0U	1U = write the test data to EEPROM at start-up. 0U = use whatever is already stored.
INITIAL_ROM_BLOCK	1U	Value written to ROM_BLOCK during provisioning
INITIAL_COPY1_DATA	67892U	Copy 1 data during provisioning
INITIAL_COPY2_DATA	12345U	Copy 2 data during provisioning (also used for the deliberately wrong Copy 1 CRC)

8.2 S32 Design Studio peripheral components

Peripheral	Instance / config used in code	Required setting
Clock	clockMan1_InitConfig0	Clock configuration that enables LPUART, LPI2C and CRC modules
Pins	g_pin_mux_InitConfigArr0 (NUM_OF_CONFIGURED_PINS 0)	LPUART TX/RX pins and LPI2C SDA/SCL pins routed to the board connectors
UART	INST_LPUART_1, lpuart_0_InitConfig0, lpUartState0	Baud rate, 8 data bits, no parity, 1 stop bit (must match PuTTY)

Peripheral	Instance / config used in code	Required setting
I2C	INST_LPI2C0, lpi2c0_MasterConfig0, lpi2cState	Master mode, baud rate supported by the EEPROM (standard 100 kHz or fast 400 kHz)
CRC	INST_CRC_1, crc_1_Cfg0	CRC width and polynomial of your choice; the same configuration is used for every calculation
OS interface	OSIF_TimeDelay()	Used for the 5 ms EEPROM write delay (OSIF component enabled)

8.3 Hardware connections

Connection	Details
EEPROM (I2C)	SDA and SCL of LPI2C0 to the EEPROM, with pull-up resistors, EEPROM at 7-bit address 0x50
UART to PC	LPUART_1 TX/RX to the PC through the board's USB-serial interface

8.4 PuTTY settings

Setting	Value
Connection type	Serial
Serial line	The COM port of the board (check in Windows Device Manager)
Speed (baud)	Same as the baud rate set in lpuart_0_InitConfig0
Data bits / Parity / Stop bits	8 / None / 1
Flow control	None

9. How to run the test

1. Create or open the project in S32 Design Studio with the components in section 8.2 and add eeprom.c / eeprom.h.
2. In main.c, set ENABLE_EEPROM_PROVISIONING to 1U.
3. Build the project and flash it to the board.
4. Open PuTTY with the settings in section 8.4, then reset the board.
5. Read the output. Provisioning runs first, then the test runs in the same boot (expected output in section 11.1).
6. Set ENABLE_EEPROM_PROVISIONING back to 0U, rebuild and flash.

7. Reset the board again and check the second output (section 11.2).

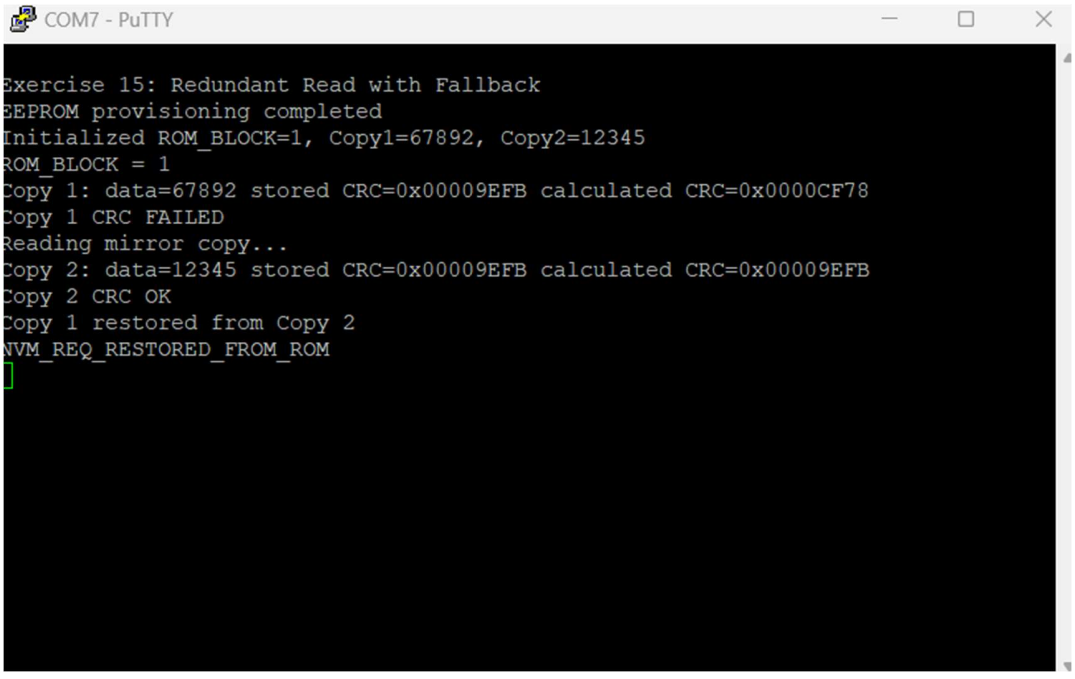
Important: the repair in step 5 is saved permanently in EEPROM. After it, Copy 1 is healthy. To see the fallback again, provision again (set the switch to `1U` and re-flash).

10. Test cases

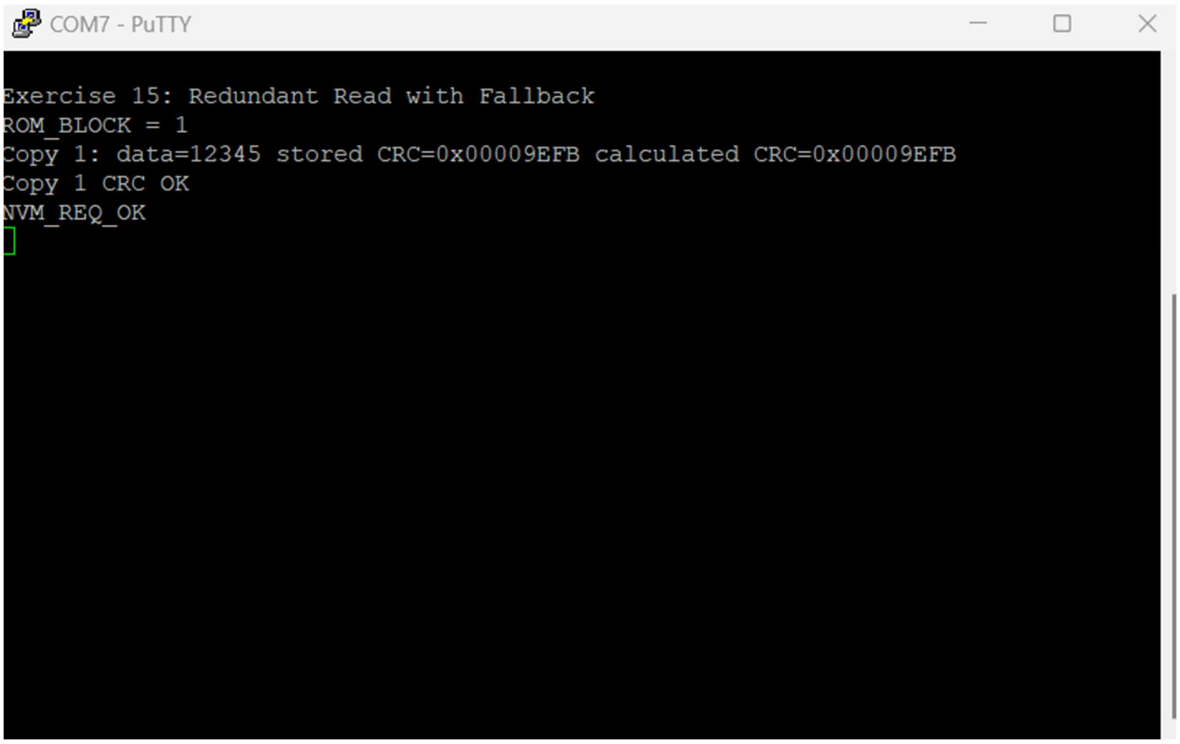
ID	Setup	Expected behaviour	Expected final status
TC1	Provisioning on, defaults (ROM_BLOCK=1, Copy 1 bad CRC, Copy 2 good)	Copy 1 CRC fails, Copy 2 CRC passes, Copy 1 is repaired	`NVM_REQ_RESTORED_FROM_ROM`
TC2	Provisioning off, run again after TC1	Copy 1 now passes its CRC check	`NVM_REQ_OK`
TC3	Provisioning on with `INITIAL_ROM_BLOCK = 0U`	Prints `ROM_BLOCK is 0`, no CRC checks	`NVM_REQ_INTEGRITY_FAILED`
TC4	Provisioning on with `INITIAL_ROM_BLOCK = 2U` (any value other than 0 or 1)	Prints `ROM_BLOCK is invalid`	`NVM_REQ_INTEGRITY_FAILED`

11. Expected UART output

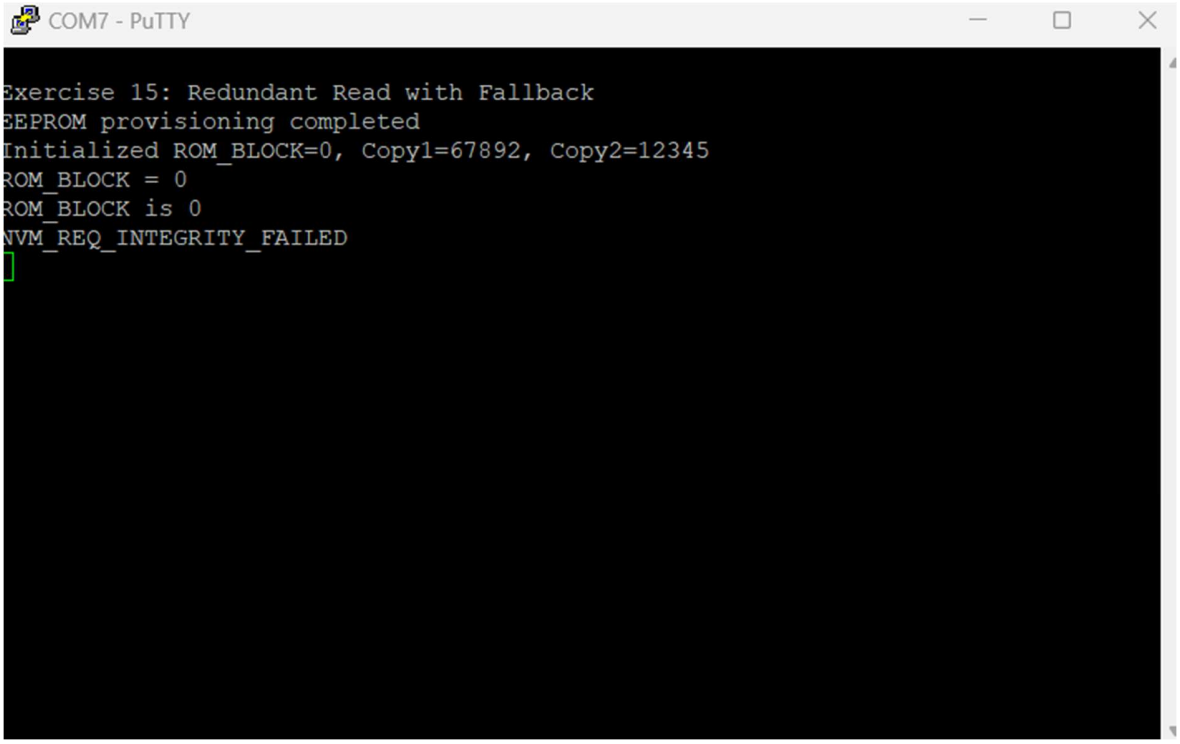
11.1 First run (TC1): provisioning on



11.2 Second run (TC2): provisioning off



11.3 ROM_BLOCK = 0 (TC3)



12. UART message reference

Message	Printed when
Exercise 15: Redundant Read with Fallback	Program start
EEPROM provisioning completed	All five provisioning writes succeeded
Initialized ROM_BLOCK, Copy1, Copy2	Right after provisioning, shows the written values
ROM_BLOCK = (0/1)	The flag was read from EEPROM
Copy 1: data, stored CRC, calculated CRC	Copy 1 was read and its CRC recalculated
Copy 2: data, stored CRC, calculated CRC	Copy 2 was read and its CRC recalculated
Copy 1 CRC OK / Copy 1 CRC FAILED	Result of the Copy 1 check
Copy 2 CRC OK / Copy 2 CRC FAILED	Result of the Copy 2 check
Reading mirror copy...	Fallback to Copy 2 has started
Copy 1 restored from Copy 2	Repair finished successfully
ERROR: ... write failed / read failed / restoration failed	An I2C read or write returned an error
EEPROM provisioning failed	A provisioning write failed; the program then halts in an endless loop

13. Notes and observations

- The status name NVM_REQ_RESTORED_FROM_ROM uses the word ROM, but in this exercise the backup is simply Copy 2 stored in the same EEPROM. ROM_BLOCK acts as a flag that allows the fallback.
- The checks for ROM_BLOCK = 0 and for any other invalid value are separate, which gives a clearer message for each case.
- After a successful repair, the code does not re-read Copy 1 to confirm it. If you want extra proof, add a second CRC check after the restore.
- CalculateCRC() re-initializes the CRC module on each call, so every result starts from the same seed value.
- The program ends in an endless loop after printing the result, so reset the board to run it again.
- Provisioning writes to the EEPROM on every boot while the switch is 1U. Set it back to 0U after the first run to avoid unnecessary writing.